

C++ Programming

1. What is the purpose of a `**vtable**` (virtual table) in C++?

- A) To manage memory allocation for derived classes.
- B) To store the addresses of all non-virtual functions.
- C) To enable run-time polymorphism by storing the addresses of virtual functions.
- D) To track the number of objects created from a class.

Answer: C) To enable run-time polymorphism by storing the addresses of virtual functions.

2. What does the keyword ``constexpr`` (C++11) primarily indicate?

- A) The variable or function can only be used with constant values.
- B) The value of the variable or function can be evaluated at compile time.
- C) The variable is stored in read-only memory.
- D) The function cannot throw exceptions.

Answer: B) The value of the variable or function can be evaluated at compile time.

3. In C++, how is a `**defaulted**` member function (C++11) declared?

- A) ``MyClass() = default;``
- B) ``MyClass() = 0;``
- C) ``MyClass() : default {}``
- D) ``default MyClass();``

Answer: A) ``MyClass() = default;``

4. Which casting operator should be used for converting between related types in an inheritance hierarchy when you are certain the object is of the derived type?

- A) ``static_cast``
- B) ``dynamic_cast``
- C) ``reinterpret_cast``
- D) ``const_cast``

Answer: A) ``static_cast``

5. What is the primary difference between **shallow copy** and **deep copy**?

A) Shallow copy duplicates all member fields; Deep copy duplicates only public fields.

B) Shallow copy duplicates the object but shares dynamically allocated data; Deep copy duplicates everything, including the pointed-to data.

C) Deep copy is done by the default copy constructor; Shallow copy requires a custom copy constructor.

D) Shallow copy uses `new`; Deep copy uses `malloc`.

Answer: B) Shallow copy duplicates the object but shares dynamically allocated data; Deep copy duplicates everything, including the pointed-to data.

6. Which STL container stores elements as key-value pairs where keys are kept unique and typically implemented using a balanced tree?

A) `std::unordered_map`

B) `std::vector`

C) `std::map`

D) `std::set`

Answer: C) `std::map`

7. The **zero-initialization** of a non-static data member (N.S.D.M.) in C++ is performed using:

A) `= 0`

B) `{}` (Braced initializer/Value initialization)

C) `()` (Parentheses initializer)

D) `nullptr`

Answer: B) `{}` (Braced initializer/Value initialization)

8. What is the use of the `std::numeric_limits<T>::max()` function?

A) It returns the maximum size of a container of type T.

B) It returns the largest finite representable value of type T.

C) It returns the maximum number of elements in an array.

D) It returns the maximum index of an STL container.

Answer: B) It returns the largest finite representable value of type T.

9. Which C++ standard introduced **move semantics**?

A) C++98

B) C++03

C) C++11

D) C++17

Answer: C) C++11

10. If a base class is inherited using **virtual inheritance**, what is the primary benefit?

A) It allows the base class to have non-virtual destructors.

B) It ensures that only one instance of the virtual base class exists in the derived object (solving the Diamond Problem).

C) It makes all members of the base class public.

D) It prevents the derived class from defining new members.

Answer: B) It ensures that only one instance of the virtual base class exists in the derived object (solving the Diamond Problem).

11. What does the `typeid` operator return?

A) The memory address of the type.

B) A reference to a `std::type_info` object for the type.

C) The size of the type in bytes.

D) A boolean indicating if the type is primitive.

Answer: B) A reference to a `std::type_info` object for the type.

12. The `std::array` container (C++11) is different from `std::vector` because:

A) `std::array` stores elements on the heap; `std::vector` stores them on the stack.

B) `std::array` has a fixed size determined at compile time; `std::vector`'s size is dynamic at run time.

C) `std::array` allows random access; `std::vector` only allows sequential access.

D) `std::array` is a linked list; `std::vector` is a dynamic array.

Answer: B) `std::array` has a fixed size determined at compile time; `std::vector`'s size is dynamic at run time.

13. What is a `**literal operator**` (C++11)?

- A) An operator that allows defining custom literals (e.g., ``10_km``).
- B) An operator used for comparing two string literals.
- C) An operator that converts any literal to a pointer.
- D) An operator that restricts a literal to a constant expression.

Answer: A) An operator that allows defining custom literals (e.g., ``10_km``).

14. The ``noexcept`` specifier is primarily used to:

- A) Guarantee that a function will not throw an exception.
- B) Force the compiler to inline a function.
- C) Prevent a function from being overloaded.
- D) Ensure that a function's return value is a constant.

Answer: A) Guarantee that a function will not throw an exception.

15. What does the ``std::future`` object represent in C++ concurrency (C++11)?

- A) A type of thread used for async operations.
- B) A communication channel for passing data between threads.
- C) A placeholder for the result of an asynchronous operation.
- D) A lock mechanism for protecting shared data.

Answer: C) A placeholder for the result of an asynchronous operation.

16. Which C++ feature allows you to initialize multiple variables from a single structure, array, or pair/tuple, introduced in C++17?

- A) Destructuring Assignment
- B) Tuple unpacking
- C) Structured Bindings
- D) Variadic Templates

Answer: C) Structured Bindings

17. What is the role of ``std::bind`` (C++11)?

- A) To dynamically link external libraries.

- B) To create a new function object (functor) with some arguments pre-bound.
- C) To bind a pointer to a specific memory location.
- D) To enforce type-safe initialization.

Answer: B) To create a new function object (functor) with some arguments pre-bound.

18. A class whose interface is defined completely in terms of another class or template is called a:

- A) Concrete class
- B) Mixin
- C) Policy class
- D) Wrapper or Adapter

Answer: D) Wrapper or Adapter

19. Which operator is implicitly overloaded to handle deep copies when one object is assigned to another (`obj1 = obj2;`)?

- A) Copy Constructor
- B) Assignment Operator (`operator=`)
- C) Move Assignment Operator
- D) Subscript Operator (`operator[]`)

Answer: B) Assignment Operator (`operator=`)

20. When should `std::shared_ptr` be preferred over `std::unique_ptr`?

- A) Always, because `shared_ptr` is safer.
- B) When the resource ownership must be shared among multiple owners.
- C) When the object is stored on the stack.
- D) When the object is passed to a C-style function.

Answer: B) When the resource ownership must be shared among multiple owners.

21. What is the name of the pointer that is pointing to a memory location that has already been deallocated?

- A) Wild pointer
- B) Null pointer
- C) Invalid pointer

D) Dangling pointer

Answer: D) Dangling pointer

22. What does the `static_assert` (C++11) declaration check?

A) An assertion that is checked at run time.

B) An assertion that is checked at compile time.

C) An assertion that is checked by the preprocessor.

D) A runtime check for memory allocation.

Answer: B) An assertion that is checked at compile time.

23. Which STL container is optimized for fast insertion and deletion at both ends?

A) `std::vector`

B) `std::list`

C) `std::deque` (Double-ended queue)

D) `std::array`

Answer: C) `std::deque` (Double-ended queue)

24. What is the use of the `std::accumulate` algorithm?

A) To sort elements in a range.

B) To find the maximum element.

C) To calculate the sum or perform a generalized reduction operation on a range of elements.

D) To merge two sorted ranges.

Answer: C) To calculate the sum or perform a generalized reduction operation on a range of elements.

25. A function template can be specialized for a specific type using which syntax?

A) `template <int> void func(int);`

B) `template <> void func<int>(int);`

C) `generic void func<int>(int);`

D) `template class func<int>;`

Answer: B) `template <> void func<int>(int);`

26. Which modifier is used with a class member function to specify that the function will not modify the state of the object?

- A) ``static``
- B) ``volatile``
- C) ``const``
- D) ``extern``

Answer: C) ``const``

27. What is the output of ``sizeof(std::string)``?

- A) Always 4 bytes.
- B) Always 8 bytes.
- C) The size of the string content plus the null terminator.
- D) Implementation-defined, typically small (e.g., 24 or 32 bytes) regardless of content length, due to Small String Optimization (SSO).

Answer: D) Implementation-defined, typically small (e.g., 24 or 32 bytes) regardless of content length, due to Small String Optimization (SSO).

28. The `**Visitor Pattern**` in C++ is often implemented to:

- A) Manage class destruction.
- B) Separate an algorithm from the object structure on which it operates.
- C) Enforce strict data encapsulation.
- D) Handle multi-threading.

Answer: B) Separate an algorithm from the object structure on which it operates.

29. Which C++ function should be used for type-safe memory allocation of arrays, ensuring each element is initialized?

- A) ``malloc()``
- B) ``new[]``
- C) ``calloc()``
- D) ``realloc()``

Answer: B) ``new[]``

30. What is the primary use of the `std::set` container?

- A) To store key-value pairs.
- B) To store a collection of unique, sorted elements.
- C) To store elements in LIFO order.
- D) To store a list of non-unique elements.

Answer: B) To store a collection of unique, sorted elements.

31. Which of the following is true about **Private Inheritance** in C++?

- A) Public and Protected members of the base class become Protected in the derived class.
- B) Public and Protected members of the base class become Private in the derived class.
- C) Only public members become private; protected members remain protected.
- D) Private inheritance is not allowed in C++.

Answer: B) Public and Protected members of the base class become Private in the derived class.

32. The concept of **Covariant Return Types** means:

- A) A derived class overriding function can return a base class pointer/reference when the base class function returns a derived class pointer/reference.
- B) A derived class overriding function can return a pointer or reference to a derived class when the base class function returns a pointer or reference to the base class.
- C) The return type must always be `void`.
- D) The return type must be the same as one of the argument types.

Answer: B) A derived class overriding function can return a pointer or reference to a derived class when the base class function returns a pointer or reference to the base class.

33. What is a **forward declaration** of a class?

- A) Defining all methods of the class before the constructor.
- B) Declaring a class without defining its members, usually using `class MyClass;`.
- C) Declaring a class as a `friend` of another class.
- D) Declaring all data members as public.

Answer: B) Declaring a class without defining its members, usually using `class MyClass;`.

34. Which operator is used for accessing an element in an STL container when using an iterator?

- A) ``->``
- B) ``[]``
- C) ``*`` (dereference)
- D) ``::``

Answer: C) ``*`` (dereference)

35. The ``std::map::at()`` method provides access to an element, but unlike ``operator[]``, it:

- A) Does not allow modification of the element.
- B) Throws an ``std::out_of_range`` exception if the key is not found.
- C) Inserts the element if the key is not found.
- D) Is only available for C++98, not C++11.

Answer: B) Throws an ``std::out_of_range`` exception if the key is not found.

36. What is the purpose of the ``std::thread::join()`` function?

- A) To pause the calling thread until the target thread finishes execution.
- B) To merge two threads into one.
- C) To detach the thread so it runs independently.
- D) To synchronize access to shared data.

Answer: A) To pause the calling thread until the target thread finishes execution.

37. Which C++ feature is used to define functions that accept an arbitrary number of arguments of arbitrary types (C++11)?

- A) Function Overloading
- B) Function Pointers
- C) Variadic Templates
- D) Friend Functions

Answer: C) Variadic Templates

38. The ``std::swap`` algorithm efficiently exchanges the values of two objects, often relying on `**move semantics**` for complex types. Why is this generally better than manual swapping?

- A) It's thread-safe.

- B) It avoids three copy operations, replacing them with cheaper move operations.
- C) It can swap objects of different types.
- D) It can swap private members directly.

Answer: B) It avoids three copy operations, replacing them with cheaper move operations.

39. What is a **C RTP** (Curiously Recurring Template Pattern)?

- A) A design pattern where a class inherits from a template specialization of itself.
- B) A recursive data structure like a linked list.
- C) A template used to generate random numbers.
- D) A template that only accepts integer arguments.

Answer: A) A design pattern where a class inherits from a template specialization of itself.

40. When a function is declared as **static** within a class, which of the following is true?

- A) It can access non-static member variables.
- B) It receives the `this` pointer as an implicit argument.
- C) It belongs to the class type itself, not to any specific object.
- D) It must be defined as an inline function.

Answer: C) It belongs to the class type itself, not to any specific object.

41. What is the C++ standard library mechanism for generating random numbers?

- A) `rand()` and `srand()` from `<cstdlib>`.
- B) The facilities in the `<random>` header (e.g., engines and distributions).
- C) The `std::random()` function in `<cmath>`.
- D) Using a special `std::RandomNumberGenerator` class.

Answer: B) The facilities in the `<random>` header (e.g., engines and distributions).

42. What does the `reinterpret_cast` operator do?

- A) Converts a derived class pointer to a base class pointer.
- B) Safely converts a number to a string.
- C) Converts a pointer of one type to a pointer of another unrelated type (low-level, dangerous cast).

D) Removes the ``const`` qualifier.

Answer: C) Converts a pointer of one type to a pointer of another unrelated type (low-level, dangerous cast).

43. Which of the following is NOT an **associative container** in the STL?

A) ``std::map``

B) ``std::set``

C) ``std::vector``

D) ``std::multiset``

Answer: C) ``std::vector``

44. What does the ``std::pair`` template allow you to store?

A) Two elements of the same type.

B) Two elements of potentially different types.

C) An unlimited number of elements.

D) Only pointers.

Answer: B) Two elements of potentially different types.

45. The C++ ``enum class`` (scoped enumeration, C++11) is preferred over plain ``enum`` because:

A) It allows multiple enums to share the same underlying type.

B) It avoids name collisions by scoping the enumerators.

C) It automatically converts enumerators to integers.

D) It is only available in C++98.

Answer: B) It avoids name collisions by scoping the enumerators.

46. When is a **Move Constructor** called?

A) When initializing an object from an rvalue (e.g., a temporary object).

B) When initializing an object from an lvalue.

C) When allocating memory using ``new``.

D) When the object goes out of scope.

Answer: A) When initializing an object from an rvalue (e.g., a temporary object).

47. What is the purpose of the `std::atomic` class template?

- A) To define complex numbers.
- B) To ensure that operations on a shared variable are thread-safe and indivisible.
- C) To replace integer types with a larger type.
- D) To prevent memory leaks in concurrent applications.

Answer: B) To ensure that operations on a shared variable are thread-safe and indivisible.

48. The `std::unique` algorithm is used to:

- A) Sort a container.
- B) Remove all duplicate consecutive elements from a sorted range.
- C) Remove all elements with a specific value.
- D) Find the first unique element.

Answer: B) Remove all duplicate consecutive elements from a sorted range.

49. What is a `pack expansion` in the context of Variadic Templates?

- A) The process of putting elements into a tuple.
- B) The syntax used to generate code for each element in a parameter pack.
- C) The process of expanding a template into multiple files.
- D) The way the compiler handles recursive template instantiation.

Answer: B) The syntax used to generate code for each element in a parameter pack.

50. What is a `perfect forwarding` (C++11)?

A) The ability of a forwarding reference to preserve the value category (lvalue or rvalue) of its argument when passing it to another function.

- B) Always passing arguments by value.
- C) Always passing arguments by constant reference.
- D) Passing only simple types between functions.

Answer: A) The ability of a forwarding reference to preserve the value category (lvalue or rvalue) of its argument when passing it to another function.

51. The `std::tuple` container is an extension of:

- A) ``std::list``
- B) ``std::vector``
- C) ``std::map``
- D) ``std::pair``

Answer: D) ``std::pair``

52. What does the ``std::async`` function (C++11) do?

- A) It starts a new operating system process.
- B) It runs a callable object asynchronously, potentially on a new thread, and returns a ``std::future``.
- C) It guarantees synchronous execution.
- D) It is a replacement for the ``new`` operator.

Answer: B) It runs a callable object asynchronously, potentially on a new thread, and returns a ``std::future``.

53. Which of the following is an example of an **Aggregate Initialization** (C++98/11)?

- A) ``MyStruct s = {1, 2.0};``
- B) ``MyClass c(5);``
- C) ``int *p = new int[10];``
- D) ``int x;``

Answer: A) ``MyStruct s = {1, 2.0};``

54. What is the purpose of the ``std::chrono::duration`` class?

- A) To manage memory usage over time.
- B) To represent a time interval.
- C) To store a specific point in time.
- D) To measure the size of a file.

Answer: B) To represent a time interval.

55. Which of the following is NOT a good practice when dealing with headers in C++?

- A) Use ``#ifndef`` guards in every header file.
- B) Include the minimum necessary headers.

- C) Use `using namespace std;` in header files.
- D) Use forward declarations when possible instead of including headers.

Answer: C) Use `using namespace std;` in header files.

56. The **Primary Template** for a class template is:

- A) The only template definition allowed.
- B) The most general definition, before any specializations.
- C) A template specialized for the `int` type.
- D) A template that only uses non-type parameters.

Answer: B) The most general definition, before any specializations.

57. When is the **Default Copy Assignment Operator** generated by the compiler?

- A) Always.
- B) Only if the programmer explicitly requests it with `= default;`.
- C) If the class does not contain a user-defined copy assignment operator, copy constructor, or destructor.
- D) Never.

Answer: C) If the class does not contain a user-defined copy assignment operator, copy constructor, or destructor.

58. What is the memory model term for the synchronization mechanism that protects shared data by only allowing one thread to access it at a time?

- A) Atomic operation
- B) Thread pool
- C) Mutex (Mutual Exclusion)
- D) Condition variable

Answer: C) Mutex (Mutual Exclusion)

59. The `std::lock_guard` class (C++11) is used for:

- A) Implementing the Observer pattern.
- B) RAII-style automatic locking and unlocking of a mutex.
- C) Manually managing thread locks.

D) Protecting an external resource from access.

Answer: B) RAIL-style automatic locking and unlocking of a mutex.

60. What does the term ****ODR**** (One Definition Rule) refer to in C++?

A) Every class must have one destructor.

B) Every function must have one return statement.

C) Any variable, function, class, or template used in a program must have exactly one definition.

D) All member functions must be defined outside the class.

Answer: C) Any variable, function, class, or template used in a program must have exactly one definition.

61. Which of the following is NOT a standard type of C++ stream?

A) `std::stringstream`

B) `std::iostream`

C) `std::networkstream`

D) `std::fstream`

Answer: C) `std::networkstream`

62. What is the output of the bitwise operation `15 & 6`? (15 is 1111, 6 is 0110)

A) 15

B) 6

C) 1

D) 4

Answer: B) 6

63. The ****Decorator Pattern**** in C++ is primarily used to:

A) Simplify the interface of a complex subsystem.

B) Dynamically add new responsibilities to an object.

C) Separate the construction of a complex object from its representation.

D) Ensure that a class has only one instance.

Answer: B) Dynamically add new responsibilities to an object.

64. What is the result of using `sizeof...()` on a parameter pack?

- A) A list of the sizes of the types in the pack.
- B) The number of elements (arguments) in the parameter pack.
- C) The total memory size occupied by the pack elements.
- D) A compile-time error.

Answer: B) The number of elements (arguments) in the parameter pack.

65. Which loop construct is best suited for an infinite loop with termination based on a condition checked in the middle of the loop body?

- A) `for (;;)` with `break`
- B) `while (true)` with `continue`
- C) `do-while` loop
- D) Range-based `for` loop

Answer: A) `for (;;)` with `break`

66. When a `**member function` is defined inside the class body`**`, it is implicitly treated as:

- A) `static`
- B) `virtual`
- C) `inline`
- D) `extern`

Answer: C) `inline`

67. What is the role of the `std::initializer_list` (C++11) class?

- A) To force all data members to be initialized in the constructor.
- B) To enable uniform initialization syntax using `{}` for various containers and objects.
- C) To initialize arrays only.
- D) To define the order of constructor calls.

Answer: B) To enable uniform initialization syntax using `{}` for various containers and objects.

68. The `**Observer Design Pattern**` in C++ relies on what primary mechanism?

- A) Multiple Inheritance
- B) A one-to-many dependency relationship between objects.
- C) Static member functions
- D) Friend classes

Answer: B) A one-to-many dependency relationship between objects.

69. What is the meaning of the array declaration `int arr[10] {};` (C++11)?

- A) An array of 10 uninitialized integers.
- B) An array of 10 integers initialized to zero (Value Initialization).
- C) A syntax error.
- D) An array whose size is determined at runtime.

Answer: B) An array of 10 integers initialized to zero (Value Initialization).

70. Which C++ header provides the facility for working with complex numbers?

- A) `<math.h>`
- B) `<complex.h>`
- C) `<cmath>`
- D) `<complex>`

Answer: D) `<complex>`

71. What does `**SFINAE**` stand for in C++ template programming?

- A) Syntax Fixes Immediately After Name Errors
- B) Substitution Failure Is Not An Error
- C) System For Interface And Native Extensibility
- D) Standard Functionality Is Now Available Everywhere

Answer: B) Substitution Failure Is Not An Error

72. Which type of STL iterator can only move forward and can be used for single-pass algorithms?

- A) Random Access Iterator
- B) Bidirectional Iterator
- C) Output Iterator

D) Input Iterator

Answer: D) Input Iterator

73. The `std::get<N>()` function is used to access elements of which type of container?

A) `std::vector`

B) `std::list`

C) `std::map`

D) `std::tuple`

Answer: D) `std::tuple`

74. What is the purpose of a **User-Defined Conversion Function**?

A) To convert an object of a class to another type (e.g., `operator int()`).

B) To implicitly call the copy constructor.

C) To define a new template specialization.

D) To convert a string to a C-style char array.

Answer: A) To convert an object of a class to another type (e.g., `operator int()`).

75. What is the primary difference between `std::set` and `std::unordered_set`?

A) `std::set` is faster for lookups.

B) `std::unordered_set` keeps elements sorted.

C) `std::unordered_set` uses a hash table for O(1) average time complexity for lookups/insertions.

D) `std::set` allows duplicate elements.

Answer: C) `std::unordered_set` uses a hash table for O(1) average time complexity for lookups/insertions.

76. Which keyword can be used to explicitly specify the underlying type of an `enum class`?

A) `type`

B) `underlying`

C) `:` (colon)

D) `class`

Answer: C) `:` (colon)

77. What is a **compile-time constant** that is known at the time of compilation?

- A) `volatile` variable
- B) `const` variable initialized with a literal.
- C) `static` variable
- D) `extern` variable

Answer: B) `const` variable initialized with a literal.

78. The `std::unique_lock` class is preferred over `std::lock_guard` when:

- A) You need to manually manage the lock/unlock operations during its lifetime (e.g., using `lock()` and `unlock()`).
- B) You need to ensure the lock is never released.
- C) You are locking an array.
- D) You only need a simple RAII-style lock.

Answer: A) You need to manually manage the lock/unlock operations during its lifetime (e.g., using `lock()` and `unlock()`).

79. What does the expression `delete [] ptr;` do?

- A) Deletes the pointer variable `ptr` itself.
- B) Deallocates memory pointed to by `ptr` that was allocated using `new` for a single object.
- C) Deallocates memory pointed to by `ptr` that was allocated using `new[]` for an array of objects.
- D) Throws an exception.

Answer: C) Deallocates memory pointed to by `ptr` that was allocated using `new[]` for an array of objects.

80. What is the core principle behind the **Adapter Pattern**?

- A) To simplify complex code.
- B) To allow the interface of one class to be used as another.
- C) To ensure thread safety.
- D) To use only template classes.

Answer: B) To allow the interface of one class to be used as another.

81. Which header provides functionality for string manipulation like converting strings to numbers (`stoi`, `stod`, etc.)?

- A) ``<cstring>``
- B) ``<stdlib.h>``
- C) ``<iostream>``
- D) ``<string>``

Answer: D) ``<string>``

82. What is the value of `i` after the following loop finishes?

<!-- end list -->

```
```cpp
int i = 0;
for (int j = 0; j < 5; j++) {
 if (j % 2 == 0) continue;
 i++;
}
```
```

- A) 5
- B) 3
- C) 2
- D) 0

Answer: C) 2

83. Which C++ feature allows you to initialize a reference member in a constructor?

- A) Explicitly using `this->ref\_member = value;`.
- B) Using the member initializer list.
- C) It is not possible to have reference members.
- D) Defining the reference as `static`.

Answer: B) Using the member initializer list.

84. What is the primary purpose of **Move Semantics**?

- A) To enable compile-time type checking.
- B) To optimize expensive copy operations by transferring resources from temporary objects.
- C) To perform run-time type identification.
- D) To support dynamic linking.

Answer: B) To optimize expensive copy operations by transferring resources from temporary objects.

85. Which STL container is typically implemented as a doubly linked list?

- A) `std::vector`
- B) `std::deque`
- C) `std::array`
- D) `std::list`

Answer: D) `std::list`

86. What is the C++17 feature that allows a conditional statement to include an initializer (e.g., for scope-limiting purposes)?

- A) Initializer List
- B) `if` with initializer
- C) Structured Bindings
- D) Lambda Expressions

Answer: B) `if` with initializer

87. What is the purpose of the **Precompiled Header** mechanism?

- A) To include header files from the standard library.
- B) To speed up compilation by storing a parsed representation of commonly used header files.
- C) To enforce the One Definition Rule.
- D) To define macros globally.

Answer: B) To speed up compilation by storing a parsed representation of commonly used header files.

88. Which operator cannot be overloaded in C++?

- A) `+` (addition)
- B) `<<` (left shift)
- C) `::` (scope resolution)
- D) `()` (function call)

Answer: C) `::` (scope resolution)

89. What is a **POD** (Plain Old Data) type?

- A) A class with virtual functions.
- B) A class or struct that is compatible with C structs and can be safely copied byte-by-byte.
- C) A class that only contains static members.
- D) An object stored on the stack.

Answer: B) A class or struct that is compatible with C structs and can be safely copied byte-by-byte.

90. The `std::stringstream` class allows you to:

- A) Read and write to a file using stream operations.
- B) Read and write to a character array (string) in memory using stream operations.
- C) Read data from the command line.
- D) Perform asynchronous I/O.

Answer: B) Read and write to a character array (string) in memory using stream operations.

91. What is the main characteristic of a **Forward Iterator**?

- A) It can move forward and backward multiple steps.
- B) It can move only one step forward.
- C) It can access elements using the subscript operator `[]`.
- D) It can be used for both reading and writing.

Answer: B) It can move only one step forward.

92. Which function from `<algorithm>` is used to change the order of elements in a range randomly?

- A) `std::shuffle()`

- B) `std::sort()`
- C) `std::rotate()`
- D) `std::unique()`

Answer: A) `std::shuffle()`

93. The concept of **Mixins** in C++ often involves:

- A) Using only friend functions.
- B) Using template inheritance (CRTP) to inject methods into a class without traditional inheritance.
- C) Using the `extern "C"` linkage.
- D) Defining a class with only private members.

Answer: B) Using template inheritance (CRTP) to inject methods into a class without traditional inheritance.

94. What is the term for defining the body of a member function outside the class definition?

- A) Inline Definition
- B) External Definition
- C) Out-of-line Definition
- D) Global Definition

Answer: C) Out-of-line Definition

95. What does the `volatile` keyword suggest to the compiler about a variable?

- A) Its value is constant.
- B) Its value can be changed by external factors (e.g., hardware or another thread), preventing optimization.
- C) It must be stored in a CPU register.
- D) It is an automatic variable.

Answer: B) Its value can be changed by external factors (e.g., hardware or another thread), preventing optimization.

96. Which header provides the standard library functions for comparing C-style strings (char arrays)?

- A) `<string>`
- B) `<iostream>`

C) `<cstring>`

D) `<cstdlib>`

Answer: C) `<cstring>`

97. The **Factory Method Pattern** is used to:

A) Ensure a class only has one instance.

B) Define an interface for creating an object, but let subclasses decide which class to instantiate.

C) Restrict class access.

D) Dynamically add new methods to a class.

Answer: B) Define an interface for creating an object, but let subclasses decide which class to instantiate.

98. What is the C++ term for the process of determining which overloaded function or operator to call at compile time?

A) Dynamic Dispatch

B) Name Lookup and Overload Resolution

C) Run-Time Type Identification (RTTI)

D) Virtual Method Invocation

Answer: B) Name Lookup and Overload Resolution

99. In the `std::map` container, the keys must be:

A) Convertible to `int`.

B) Hashable (for good performance).

C) Copyable and Movable.

D) Comparable using an ordering relationship (e.g., `<`).

Answer: D) Comparable using an ordering relationship (e.g., `<`).

100. What is the core characteristic of an **Abstract Base Class (ABC)**?

A) It must have at least one static member.

B) It can be instantiated directly.

C) It contains at least one pure virtual function and cannot be instantiated directly.

D) It only contains private members.

Answer: C) It contains at least one pure virtual function and cannot be instantiated directly.